Pre Proyecto 3

Actividad formativa: Optimización Extremal (Extremal Optimization, EO)

El propósito de esta actividad es aplicar una metodología de desarrollo modular a la implementación de una metaheurística inspirada en sistemas complejos y la dinámica evolutiva, de manera que lo aprendido pueda posteriormente utilizarse en problemas de optimización combinatoria más complejos.
El problema de entrenamiento será el **Problema de la Mochila 0/1 (Knapsack Problem, KP)** y se utilizará la **Optimización Extremal (Extremal Optimization, EO / τ-EO)** para buscar soluciones de alta calidad que maximicen el beneficio total respetando la restricción de capacidad.
La actividad pone énfasis en **comprender, modularizar, probar y analizar** la solución utilizando **Python** junto a las bibliotecas **NumPy, Pandas, Sys y Time**.

1. Problema

Dado un conjunto de N elementos, donde cada elemento $i \in \{1, \ldots, N\}$ posee un peso $w_i > 0$ y un valor o beneficio $v_i > 0$, y una mochila con una capacidad máxima $C > 0$, se desea determinar qué elementos incluir en la mochila para:

- **Maximizar el valor total** de los elementos seleccionados: $\sum_{i=1}^{N} v_i x_i$
- **Sujeto a la restricción de capacidad**: $\sum_{i=1}^{N} w_i x_i \leq C$
- Donde $x_i \in \{0, 1\}$ representa la inclusión ($x_i = 1$) o exclusión ($x_i = 0$) del elemento i.

El programa deberá utilizar **Extremal Optimization (τ-EO)** para modelar la mochila como un ecosistema donde cada elemento representa una especie con un nivel de adaptación individual (*fitness*), reemplazando iterativamente a las especies menos adaptadas para guiarse hacia configuraciones óptimas globales.
El objetivo es implementar correctamente la metodología modular, encontrar la solución óptima o cercana a la óptima para las instancias de prueba, y analizar el comportamiento dinámico del algoritmo.

2. Forma de ejecución

El programa deberá ejecutarse desde la línea de comandos (CLI) indicando los parámetros requeridos. Por ejemplo:

```
python eo.py instancia_mochila.txt 1000 1.4 12345 resultados.csv
```

Los parámetros deberán permitir, como mínimo, indicar:

- **Archivo de entrada**: ruta de la instancia del problema de la mochila (pesos, valores y capacidad).
- **Número de iteraciones / condición de término**: total de pasos evolutivos del ecosistema.
- **Valor de Tau (τ)**: parámetro de escala para la distribución de probabilidad de selección.
- **Semilla aleatoria (*seed*)**: valor entero para asegurar la reproducibilidad del generador estocástico.
- **Archivo de salida**: ruta del archivo estructurado donde se guardarán los resultados.

Salida

Los resultados deberán almacenarse en un archivo estructurado (por ejemplo CSV) utilizando la biblioteca **Pandas**, para su posterior análisis o visualización.
El archivo deberá contener información suficiente para analizar los experimentos, incluyendo:

- Instancia evaluada
- Semilla utilizada
- Valor del parámetro τ
- Número de iteración
- Valor total (beneficio) de la mejor mochila encontrada
- Peso total consumido por la mejor solución
- Indicador de validez (cumple o excede la capacidad C)

- Tiempo total de ejecución (usando el módulo `time`)

La generación de gráficos no forma parte obligatoria del programa principal, pero puede desarrollarse mediante un script independiente (ej. `graficar.py`).

3. Organización del programa

La solución deberá aplicar una separación estricta y sencilla de responsabilidades.
Como mínimo, se deberán utilizar **dos archivos principales**:

```
eo.py
operadores.py
```

eo.py

Este archivo deberá contener la **función principal del algoritmo Extremal Optimization**.
La lectura del programa debe permitir reconocer con facilidad las etapas principales del algoritmo evolutivo:

```
inicializar ecosistema (mochila)
          ↓
evaluar fitness individual de cada especie/elemento
          ↓
ordenar especies por ranking de adaptabilidad
          ↓
seleccionar especie a modificar (regla de la ruleta / potencia con τ)
          ↓
reemplazar/mutar estado de la especie seleccionada
          ↓
evaluar fitness global del ecosistema y actualizar mejor solución
          ↓
repetir hasta cumplir condición de término
```

El archivo `eo.py` debe importar y utilizar las funciones implementadas en `operadores.py`.

operadores.py

Este archivo funcionará como una **biblioteca de operadores genéricos y específicos** del ecosistema/mochila.
Aquí deberán implementarse las funciones necesarias sugeridas en la especificación base:

- **Inicialización del ecosistema**: generación del estado inicial de la mochila ($x \in \{0,1\}^N$).
- **Evaluación del fitness individual**: cálculo de la adaptabilidad q_i de cada especie/elemento (basado en la eficiencia de beneficio/peso v_i/w_i y el estado de restricción).
- **Selección de especie (Ruleta por ranking / τ-EO)**: ordenamiento por ranking $k \in \{1, \ldots, N\}$ de peor a mejor y selección probabilística proporcional a $k^{-\tau}$.
- **Reemplazo/mutación de la especie**: operador de cambio de estado de la especie seleccionada ($x_i \leftarrow 1 - x_i$).
- **Evaluación del ecosistema**: cálculo de la función de evaluación global (beneficio total con penalización proporcional al exceso de capacidad).
- **Otras funciones auxiliares**: lectura de instancias, control de tiempo con `time` y manejo de estructuras de datos con `NumPy` y `Pandas`.

Otros archivos

Se pueden incorporar archivos adicionales según sea necesario:

```
tests.py
graficar.py
experimentos.py
```

4. Principio de modularización

La modularización debe utilizarse para **simplificar y facilitar la comprensión del programa**.
La función principal en `eo.py` no debe contener los detalles técnicos de bajo nivel ni manipulación directa de arreglos.

Por ejemplo, la estructura conceptual principal en `eo.py` debe ser legible de la siguiente manera:

```
solucion_actual = inicializar_ecosistema(instancia, semilla)
mejor_solucion = evaluar_y_actualizar_mejor(solucion_actual)

for iteracion in range(num_iteraciones):
    fitness_especies = evaluar_fitness_especies(solucion_actual, instancia)

    especie_seleccionada = seleccionar_especie_tau(fitness_especies, tau)

    solucion_actual = reemplazar_especie(solucion_actual, especie_seleccionada)

    mejor_solucion = evaluar_y_actualizar_mejor(solucion_actual, mejor_solucion)
```

La premisa central es:
`eo.py` explica *qué* hace el algoritmo y `operadores.py` explica *cómo* se realizan sus operaciones.
El código debe permitir crear una biblioteca reusable de métodos y funciones para futuros proyectos de optimización.

5. Pruebas

Antes de ejecutar la batería de experimentos finales, se deberán verificar los principales operadores del programa mediante casos de prueba sencillos (`tests.py`).
Las pruebas deberán comprobar, al menos:

- Que los archivos de entrada de la mochila se leen e interpretan correctamente.
- Que el fitness de cada especie (q_i) asigna un menor puntaje a elementos ineficientes o que provocan sobrepeso.
- Que el mecanismo de selección por ranking respeta la ley de potencia impulsada por el parámetro τ.
- Que la función de reemplazo (*bit-flip*) altera correctamente la configuración de la solución.
- Que el algoritmo preserva siempre la **mejor mochila válida** encontrada a lo largo de las iteraciones.

6. Experimentación

Una vez validada la implementación, se deberán llevar a cabo experimentos sistemáticos modificando los parámetros del algoritmo. Para garantizar la comparabilidad, se usará una semilla controlada.
Se deben analizar e interpretar preguntas clave como:

- ¿Cómo influye la variación del valor de Tau (τ) en el balance entre exploración y explotación? (por ejemplo, evaluando $\tau \in [1.0, 3.0]$).
- ¿Cómo evoluciona la calidad de la solución al aumentar el número de iteraciones?
- ¿Qué impacto tiene la condición inicial del ecosistema según distintas semillas aleatorias?
- ¿Cuál es la relación entre el tamaño de la instancia (N), la calidad de la mochila y el tiempo de ejecución de la biblioteca?

Los datos guardados en el archivo CSV deberán ser procesados o graficados mediante scripts como `graficar.py` (ej. `python graficar.py resultados.csv`).

7. Transferencia a un problema más complejo

Al finalizar la actividad, el estudiante debe reflexionar sobre la versatilidad del método de Optimización Extremal para adaptarse a otros problemas combinatorios. Por ejemplo, al **Permutation Flow Shop Scheduling Problem (PFSP)**.
Explique brevemente en el informe/README:

1. ¿Qué componentes del flujo principal en `eo.py` se mantendrían idénticos?
2. ¿Qué funciones en `operadores.py` requieren cambio o rediseño según el nuevo problema?
3. ¿Cómo se definiría la adaptabilidad individual (*fitness* de cada especie) en el problema elegido?
4. ¿Cómo cambia el operador de reemplazo para mantener soluciones factibles?

---

8. Estructura sugerida

Se propone la siguiente organización modular del proyecto:

```
trabajo_programacion_3/
│
├── eo.py
├── operadores.py
├── tests.py
├── graficar.py
│
├── datos/
│   └── mochila_instancia1.txt
│
├── resultados/
│   └── resultados.csv
│
├── graficos/
│   └── grafico_convergencia_tau.pdf
│
└── README.md
```

9. Entrega (Actividad Formativa)

La entrega deberá contener:

- código fuente;
- archivos de entrada utilizados;
- archivos de resultados;
- casos de pruebas;
- programa de generación de gráficos, si corresponde;
- README.md con instrucciones de ejecución;

10. Consideraciones Finales

Se enfatiza el principio **KISS (Keep It Simple, Stupid)**: preferir una arquitectura limpia, legible y modular sobre abstraer en exceso.
Hiperingeniería: Se reitera evitar la complejidad innecesaria. Diseños sobreestructurados, patrones de diseño mal aplicados o abstracciones superfluas que dificulten el entendimiento y la mantención del código afectarán negativamente la evaluación del proyecto.